Experiment # 1

Introduction to C++ Basics

1. Objective/s:

- What is C++ Language
- Compiler
- Console Program
- Program Structure
- Data Types
- Scope of Variables
- Order of Precedence
- Expressions
- Type Conversion
- Examples
- Lab Task

2. What is C/C++ Language

C is a high-level and general-purpose programming language that is ideal for developing firmware or portable applications. Originally intended for writing system software, C was developed at Bell Labs by Dennis Ritchie for the Unix Operating System in the early 1970s.

Ranked among the most widely used languages, C has a compiler for most computer systems and has influenced many popular languages – notably C++.

3. Compiler

Computers understand only one language and that language consists of sets of instructions made of ones and zeros. This computer language is called *machine language*.

A single instruction to a computer could look like this:

00000	10011110

A particular computer's machine language program that allows a user to input two numbers, adds the two numbers together, and displays the total could include these machine code instructions:

00000	10011110
00001	11110100
00010	10011110

00011	11010100
00100	10111111
00101	00000000

As you can imagine, programming a computer directly in machine language using only ones and zeros is very tedious and error prone. To make programming easier, high level languages have been developed. High level programs also make it easier for programmers to inspect and understand each other's programs easier.

This is a portion of code written in C++ that accomplishes the exact same purpose:

```
int a, b, sum;

cin >> a;
cin >> b;

sum = a + b;
cout << sum << endl;
```

Even if you cannot really understand the code above, you should be able to appreciate how much easier it will be to program in the C++ language as opposed to machine language.

Because a computer can only understand machine language and humans wish to write in high level languages high level languages have to be re-written (translated) into machine language at some point. Special programs called compilers, interpreters, or assemblers that are built into the various programming applications do this.

C++ is designed to be a compiled language, meaning that it is generally translated into machine language that can be understood directly by the system, making the generated program highly efficient. For that, a set of tools are needed, known as the development toolchain, whose core are a compiler and its linker.

4. Console Program

Console programs are programs that use text to communicate with the user and the environment, such as printing text to the screen or reading input from a keyboard.

Console programs are easy to interact with, and generally have a predictable behavior that is identical across all platforms. They are also simple to implement and thus are very useful to learn the basics of a programming language: The examples in these tutorials are all console programs.

The way to compile console programs depends on the particular tool you are using

The easiest way for beginners to compile C++ programs is by using an Integrated Development Environment (IDE). An IDE generally integrates several development tools, including a text editor and tools to compile programs directly from it.

Here you have instructions on how to compile and run console programs using different free Integrated Development Interfaces (IDEs):

IDE	Platform	Console programs
Code::blocks	Windows/Linux/MacOS	Compile console programs using Code::blocks
Visual Studio Express	Windows	Compile console programs using VS Express 2013
Dev-C++	Windows	Compile console programs using Dev-C++

5. Structure of a Program

The best way to learn a programming language is by writing programs. Typically, the first program beginners write is a program called "Hello World", which simply prints "Hello World" to your computer screen. Although it is very simple, it contains all the fundamental components C++ programs have:

```
1 // my first program in C++
2 #include <iostream>
3
4 using namespace std;
5
6 int main()
7 {
8 cout<<"Hello World!";
9 }
```

```
Hello World!
```

The left panel above shows the C++ code for this program. The right panel shows the result when the program is executed by a computer. The grey numbers to the left of the panels are line numbers to make discussing programs and researching errors easier. They are not part of the program.

Let's examine this program line by line:

Line 1: `// my first program in C++`

Two slash signs indicate that the rest of the line is a comment inserted by the programmer but which has no effect on the behavior of the program. Programmers use them to include short explanations or observations concerning the code or program. In this case, it is a brief introductory description of the program.

Line 2: `#include <iostream>`

Lines beginning with a hash sign (`#`) are directives read and interpreted by what is known as the *preprocessor*. They are special lines interpreted before the compilation of the program itself begins. In this case, the directive `#include <iostream>`, instructs the preprocessor to include a section of standard C++ code, known as *header iostream*, that allows to perform standard input and output operations, such as writing the output of this program (`Hello World`) to the screen.

Line 3 and 5: A blank line.

Blank lines have no effect on a program. They simply improve readability of the code.

6.

7. Line 4: using namespace std in c++

cin and cout are declared in the header file iostream, but within std namespace. To use cin and cout in a program, use the following two statements:

```
#include <iostream>
using namespace std;
```

Line 6: `int main ()`

This line initiates the declaration of a function. Essentially, a function is a group of code statements which are given a name: in this case, this gives the name "main" to the group of code statements that follow. Functions will be discussed in detail in a later chapter, but essentially, their definition is introduced with a succession of a type (`int`), a name (`main`) and a pair of parentheses (`()`), optionally including parameters.

The function named `main` is a special function in all C++ programs; it is the function called when the program is run. The execution of all C++ programs begins with the `main` function, regardless of where the function is actually located within the code.

Lines 7 and 9: `{` and `}`

The open brace (`{`) at line 5 indicates the beginning of `main`'s function definition, and the closing brace (`}`) at line 7, indicates its end. Everything between these braces is the function's body that defines what happens when `main` is called. All functions use braces to indicate the beginning and end of their definitions.

Line 8: `cout<<"Hello World!";`

This line is a C++ statement. A statement is an expression that can actually produce some effect. It is the meat of a program, specifying its actual behavior. Statements are executed in the same order that they appear within a function's body.

This statement has two parts: First, `printf`, which is a function used to print something on screen. Second, a sentence within quotes ("Hello world!"), is the content inserted into the standard output.

Notice that the statement ends with a semicolon (`;`). This character marks the end of the statement, just as the period ends a sentence in English. All C++ statements must end with a semicolon character. One of the most common syntax errors in C++ is forgetting to end a statement with a semicolon.

You may have noticed that not all the lines of this program perform actions when the code is executed. There is a line containing a comment (beginning with `//`). There is a line with a directive for the preprocessor (beginning with `#`). There is a line that defines a function (in this case, the `main` function). And, finally, a line with a statements ending with a semicolon (the insertion into `cout`), which was within the block delimited by the braces (`{ }`) of the `main` function.

The program has been structured in different lines and properly indented, in order to make it easier to understand for the humans reading it. But C++ does not have strict rules on indentation or on how to split instructions in different lines. For example, instead of

```
int main ()
{
cout<<" Hello World!";
}
```

We could have written:

```
int main () { cout<<" Hello World!";}
```

all in a single line, and this would have had exactly the same meaning as the preceding code.

In C++, the separation between statements is specified with an ending semicolon (`;`), with the separation into different lines not mattering at all for this purpose. Many statements can be written in a single line, or each statement can be in its own line. The division of code in different lines serves only to make it more legible and schematic for the humans that may read it, but has no effect on the actual behavior of the program.

Now, let's add an additional statement to our first program:

```
// my second program in C++
#include <iostream>

using namespace std;

int main ()
{
printf(" Hello World!");
printf("I'm a C++ program ");
}
```

```
Hello World! I'm a C++ program
```

Preprocessor directives (those that begin by `#`) are out of this general rule since they are not statements. They are lines read and processed by the preprocessor before proper compilation

begins. Preprocessor directives must be specified in their own line and, because they are not statements, do not have to end with a semicolon (;).

8. Program Structure/Basics

9. Inputs/Outputs

Values of variables can be read from users and values of variables can be output to users.In between values are manipulated. In order to manipulate data, it must be loaded into memory. There are two steps to load data into memory:

- Instruct computer to allocate memory
- Include statements to put data into memory

10. Constants/Variables

- Constants

Memory location whose content can't change during execution are called Constants

```
const dataType identifier = value;
```

In C++, *const* is a reserved word

- Variable

Memory location whose content may change during execution

```
dataType identifier, identifier, . . .;
```

10.1.1. Initializing and Declaring Variables

Variables can be initialized when declared, or can be initialized in separate command line:

int first=13, second=10;

charch=' ';

double x=12.6;

10.1.2. cin>>/cout<<

cin is used with >> to gather input. The stream extraction operator is >> e.g. if miles is a double variable

- cin>> miles;

This causes computer to get a value of type double and places it in the variable miles. Using more than one variable in cin allows more than one value to be read at a time. For example, if feet and inches are variables of type int, a statement such as:

- cin>> feet >> inches;

Inputs two integers from the keyboard and places them in variables feet and inches respectively. To summarize this there are two ways to initialize a variable:

intfeet;

feet = 35; (By using the assignment statement)

cin>> feet; (By using a read statement)

cout prints the expression evaluated and its value at the current cursor position on the screen/console. The stream insertion operator is << and the syntax of cout and << is

```
cout << expression or manipulator << expression or manipulator...;
```

11. Data Types

While writing program in any language, you need to use various variables to store various information. Variables are nothing but reserved memory locations to store values. This means that when you create a variable you reserve some space in memory.

You may like to store information of various data types like character, wide character, integer, floating point, double floating point, boolean etc. Based on the data type of a variable, the operating system allocates memory and decides what can be stored in the reserved memory.

11.1. Primitive Built-in Types

C++ offers the programmer a rich assortment of built-in as well as user defined data types. Following table lists down seven basic C++ data types

Type	Keyword
Boolean	Bool

Character	Char
Integer	Int
Floating point	Float
Double floating point	Double
Valueless	Void
Wide character	wchar_t

Several of the basic types can be modified using one or more of these type modifiers

- signed
- unsigned
- short
- long

The following table shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

Type	Typical Bit Width	Typical Range
Char	1byte	-127 to 127 or 0 to 255
unsigned char	1byte	0 to 255
signed char	1byte	-127 to 127
Int	4bytes	-2147483648 to 2147483647
unsigned int	4bytes	0 to 4294967295

signed int	4bytes	-2147483648 to 2147483647
short int	2bytes	-32768 to 32767
unsigned short int	Range	0 to 65,535
signed short int	Range	-32768 to 32767
long int	4bytes	-2,147,483,648 to 2,147,483,647
signed long int	4bytes	same as long int
unsigned long int	4bytes	0 to 4,294,967,295
Float	4bytes	+/- 3.4e +/- 38 (~7 digits)
Double	8bytes	+/- 1.7e +/- 308 (~15 digits)
long double	8bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	2 or 4 bytes	1 wide character

The size of variables might be different from those shown in the above table, depending on the compiler and the computer you are using.

Following is the example, which will produce correct size of various data types on your computer.

11.2. typedef Declarations

You can create a new name for an existing type using **typedef**. Following is the simple syntax to define a new type using typedef

```
typedef type newname;
```

For example, the following tells the compiler that feet is another name for int

```
typedef int feet;
```

Now, the following declaration is perfectly legal and creates an integer variable called distance

```
feet distance;
```

12. Scope of Variables

A scope is a region of the program and broadly speaking there are three places, where variables can be declared

- Inside a function or a block which is called local variables,
- In the definition of function parameters which is called formal parameters.
- Outside of all functions which is called global variables.

We will learn what a function is and it's parameter in subsequent chapters. Here let us explain what local and global variables are.

12.1. Local Variables

Variables that are declared inside a function or block are local variables. They can be used only by statements that are inside that function or block of code. Local variables are not known to functions outside their own. Following is the example using local variables

```
#include <iostream>
using namespace std;

int main () {
   // Local variable declaration:
   int a, b;
   int c;

   // actual initialization
   a = 10;
   b = 20;
   c = a + b;

   cout << c;

   return 0;
}
```

12.2. Global Variables

Global variables are defined outside of all the functions, usually on top of the program. The global variables will hold their value throughout the life-time of your program.

A global variable can be accessed by any function. That is, a global variable is available for use throughout your entire program after its declaration. Following is the example using global and local variables −

```
#include <iostream>
using namespace std;

// Global variable declaration:
int g;

int main () {
   // Local variable declaration:
   int a, b;

   // actual initialization
   a = 10;
   b = 20;
   g = a + b;

   cout << g;

   return 0;
}
```

A program can have same name for local and global variables but value of local variable inside a function will take preference. For example

```
#include <iostream>
using namespace std;

// Global variable declaration:
int g = 20;

int main () {
   // Local variable declaration:
   int g = 10;

   cout << g;

   return 0;
}
```

When the above code is compiled and executed, it produces the following result −

```
10
```

12.3. Initializing Local and Global Variables

When a local variable is defined, it is not initialized by the system, you must initialize it yourself. Global variables are initialized automatically by the system when you define them as follows

Data Type	Initializer
Int	0
Char	'\0'
float	0
double	0
pointer	NULL

It is a good programming practice to initialize variables properly, otherwise sometimes program would produce unexpected result.

13. Order of Precedence

Order of Precedence is the principal by which order of execution of an arithmetic expression with different operators is decided. C++ has multiple arithmetic operators which include

- + addition
- - subtraction
- * multiplication
- / division
- % modulus operator

All the above mentioned operators except for % can be used with integral and floating-point data types

Any expression (explained later in manual) having multiple operators e.g. 3 * 7 - 6 + 2 * 5 / 4 + 6 will be executed by following these points

- All operations inside of () are evaluated first
- *, /, and % are at the same level of precedence and are evaluated next

- + and – have the same level of precedence and are evaluated last

Hence the example expression 3 * 7 - 6 + 2 * 5 / 4 + 6 becomes (((3 * 7) – 6) + ((2 * 5) / 4)) + 6

14. Expressions

An expression is a sequence of operators and operands that specifies a computation.

We will learn about Intergral/floating or mixed expression

14.1. Integral/Floating Expression

- If all operands are integers
  - Expression is called an integral expression
    - Yields an integral result
    - Example: 2 + 3 * 5
- If all operands are floating-point
  - Expression is called a floating-point expression
    - Yields a floating-point result
    - Example: 12.8 * 17.5 - 34.50

14.2. Mixed Expression

- Mixed expression:
  - Has operands of different data types
  - Contains integers and floating-point
- Examples of mixed expressions:

  2 + 3.5

  6 / 4 + 3.9

  5.4 * 2 – 13.6 + 18 / 2

14.3. Evaluation Rule

- If operator has same types of operands
  - Evaluated according to the type of the operands
- If operator has both types of operands
  - Integer is changed to floating-point
  - Operator is evaluated
  - Result is floating-point

- Entire expression is evaluated according to precedence rules

15. Type Conversion (Casting)

15.1. Implicit Type Conversion

- when value of one type is automatically changed to another type

15.2. Explicit Type Conversion

- provides explicit type conversion

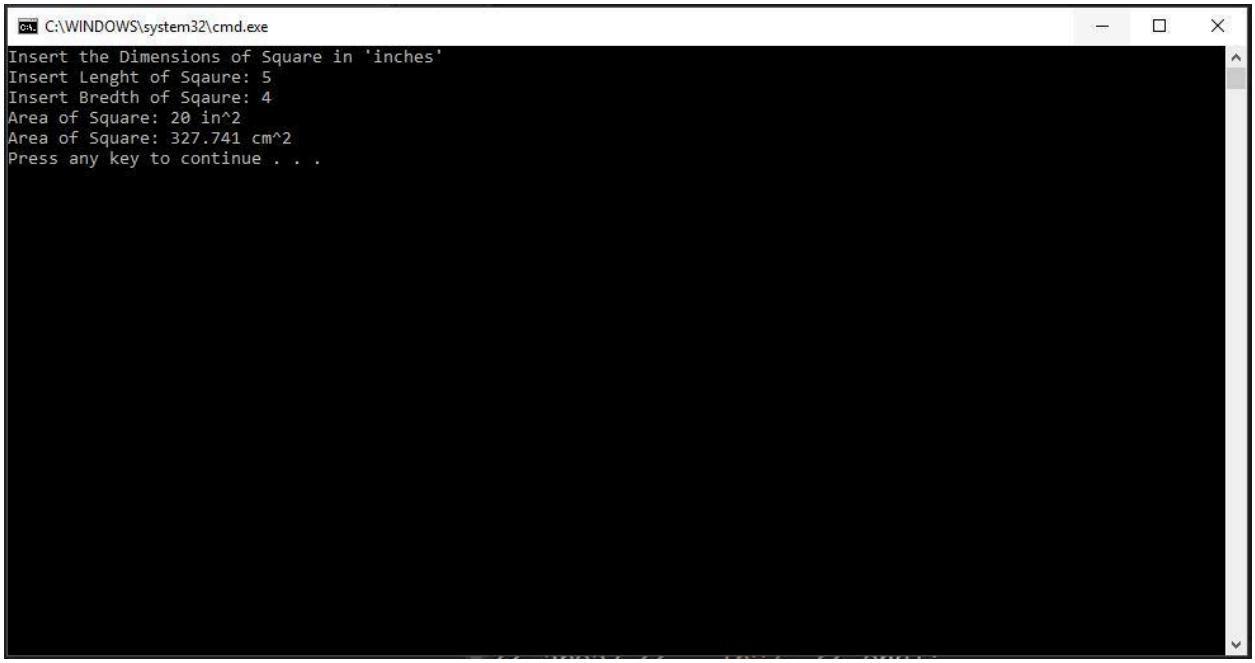
static_cast<desired dataTypeName>(expression)

EXAMPLE 2-9

Expression	Evaluates to
`static_cast<int>(7.9)`	`7`
`static_cast<int>(3.3)`	`3`
`static_cast<double>(25)`	`25.0`
`static_cast<double>(5+3)`	`= static_cast<double>(8) = 8.0`
`static_cast<double>(15) / 2`	`= 15.0 / 2` (because `static_cast<double>(15) = 15.0`) `= 15.0 / 2.0 = 7.5`
`static_cast<double>(15 / 2)`	`= static_cast<double>(7)` (because `15 / 2 = 7`) `= 7.0`
`static_cast<int>(7.8 + static_cast<double>(15) / 2)`	`= static_cast<int>(7.8+7.5)` `= static_cast<int>(15.3)` `= 15`
`static_cast<int>(7.8 + static_cast<double>(15 / 2))`	`= static_cast<int>(7.8 + 7.0)` `= static_cast<int>(14.8)` `= 14`

16. Tasks

1. Write and run a program to print your first name on the first line, your middle name on the second line and your last name on the third line using only “cout” statement.
2. Write a program in which user enters the dimensions or a square that is its length and breadth in inches. Both values must be int type. User have to calculate the area of the square $in^2$and cm^2display it to the user. Console must have following output on execution.



Note that each line in the code must have proper comments and heading of the program must also have a comment mentioning your registration number in it. Use ***const***as well if necessary.

3. Type following commands and answer each question
    a. *cout<<static_cast<double>(15/2);*
        i. Why the result is not double value although we have converted it into double.
    b. cout<<static_cast<double>(15)/2;
        i. Why now the result is double value? What is the difference between command a and b?

4. Write a Program in which user enters a number (the number can be floating or int). if the number is floating it is converted into integer and printed back on console. If the number is already integer, it is printed on console as it is.
5. Write a program in which user enters an integer value and the program divides the value by 2 and print the result in decimal format